

Rapport de stage

Développement Web

Timéo Pérou
BUT2 MMI - Avril / Juin 2025



Remerciements

Je souhaiterais remercier en premier lieu **Benjamin Lecourtier** et **Johanna Hemour**, pour avoir accepté de travailler avec moi, et donc de m'avoir fait confiance et accompagné tout au long de ce stage.

De plus, je remercie aussi les professeurs en MMI **Anthony Wright** ainsi que **Christophe Courtin** pour nous avoir accompagnés.

Et merci à mes camarades de MMI **Mathieu, Bastien, Benjamin, Tom, Evan** ainsi qu'à mon ami **Aloys**, pour m'avoir conseillé et aidé durant ce stage.

Sommaire

Introduction	4
Structure	5
Missions	
Demande initiale	7
Pré-production	8
Maquettes	9
Modélisation des données	10
Intégration	11
Gestion des données	13
Mails	17
Création de PDF	18
Conclusion	19
Glossaire	20
Annexes	22

Introduction

01

En fin de 2e année de BUT MMI, nous devions réaliser un stage de 8 semaines, du 22 Avril au 20 Juin. J'ai effectué le mien chez LACVI, une cave à vins locale, qui voulait obtenir un système de commande en ligne pour leurs clients, en particulier les associations qui ne peuvent pas venir aux horaires du magasin.

J'ai choisi ce stage car la mission correspond à une demande typique en développement web, ce qui m'offre la possibilité de tester les compétences acquises durant mes années de BUT ainsi que de les approfondir. De plus, connaissant personnellement l'entreprise, ce stage m'a permis d'aider des commerçants locaux, qui est un point qui me tient à coeur.

Structure

02

LACVI est une cave à vins situé à Beaumont en Haute-Savoie, entièrement dédiée aux vignerons indépendants de France. Ils proposent évidemment du vin, mais aussi des champagnes, spiritueux et tireuses à bière, pour les particuliers comme pour des événements.

L'entreprise comporte 2 salariés polyvalents qui s'occupent de la vente, des dégustations, de la communication ou des livraisons, du Mardi au Samedi.

Structure

La cave n'étant pas destinée à être un lieu de travail bureautique en premier lieu, ce stage s'est déroulé en hybride :

- À la fois présentiel pour montrer l'avancée du projet ou garder un contact avec le client
- Mais aussi en télétravail quand la cave était fermée ou que les tâches en cours n'étaient pas adaptées au présentiel

Nous avons déjà expérimenté les deux cadres de travail pendant les cours et surtout le semestre 4 DevWeb, donc j'étais déjà familier à ces deux types de travail, et à comment garder contact avec le client, grâce à la semaine du Web.



Intérieur de la cave

Missions

Demande initiale

03

La mission principale de ce stage était de concevoir un système de commande en ligne sans paiements.

Parmi les clients de l'entreprise, il y a beaucoup d'associations qui organisent des événements et commandent en grosses quantités. Mais vu que ses structures sont animées très souvent par des bénévoles pendant leur temps libre, peu ont l'occasion de réserver pendant les jours d'ouverture de la cave.

L'objectif de ce système est donc d'offrir une alternative aux associations de pouvoir effectuer une commande n'importe quand. Vu que le paiement s'effectue après l'événement en fonction de ce qui a été consommé, il n'est pas nécessaire d'implémenter un paiement au site.

Pré-production

J'ai d'abord commencé par planifier les différentes étapes dans la réalisation du système de commande sur Notion, en faisant une liste de tout ce qu'il y avait à faire, et en leur donnant une date arbitraire pour respecter la **méthode Agile*** de gestion de projet.

→ Planning des tâches - Annexes

La première semaine du stage était focalisée sur la pré-production, avec la réception de la demande du client et l'analyse de l'existant, qui ont abouti sur la rédaction du cahier des charges. Ainsi, la demande et les besoins du site sont clairement définis dès le début.

Le site doit permettre une navigation fluide, agréable et adaptée aux cibles, en plus de respecter la charte graphique de l'entreprise.

→ Cahier des charges - Annexes

Maquettes

Dans cette partie du projet, il fallait ensuite réaliser l'**arborescence*** du site.

J'ai choisi de le construire en parallèle de la **maquette***, car dans de précédents projets, je l'avais réalisé avant ou après et il y avait eu des problèmes quant à la cohérence du projet.

De cette manière, si une page était trop inaccessible, je pouvais le voir rapidement pendant le design des maquettes, et modifier directement en conséquences.

Cette méthode s'est avérée efficace dans la cohérence des maquettes, mais l'arborescence a quand même été modifiée par la suite, donc ça n'a pas été complètement efficace.

→ Arborescence - Annexes

Le cahier des charges validé, l'étape suivante est de designer les maquettes du futur site, côté client, admin, en mobile ou PC.

La difficulté était de proposer un site qui soit adapté aux cibles mais aussi qui plaise à l'entreprise tout en respectant sa charte graphique. Et contrairement aux autres projets en MMI, cette fois-ci j'étais seul sur les maquettes, et donc je ne pouvais pas avoir de retours directs ou de conseils, que d'ordinaire je recevais.

Ma démarche était d'abord de récupérer tous les éléments de la charte graphique de LACVI, pour commencer à designer avec. Ensuite, en reprenant des designs ou des **wireframes*** de sites marchands, trouvés suite à l'analyse de l'existant, j'ai été en mesure de les terminer assez vite.

Au final, j'ai pu proposer à la fin de la deuxième semaine des maquettes qui ont été validées par le client, et qui semblaient suffisamment claires pour la cible.

→ Maquettes - Annexes

Modélisation des données

Vu que le système de commande comporte des utilisateurs avec un **CRUD*** et des articles, cela implique la présence obligatoire d'une base de données.

Le **MCD*** est d'abord construit à partir des fonctionnalités du site, déterminées à partir des maquettes. Puis il permet de créer un **MPD*** exploitable sur le site.

LACVI utilise un service nommé Vitisoft, qui gère les stocks, les commandes auprès des viticulteurs ou la création de devis. Dans le cahier des charges figurait la possibilité de lier une **API*** de ce service au système pour plus de praticité dans l'import des articles, la création de devis et des clients.

Je me suis renseigné sur le service et j'ai contacté Vitisoft pour savoir si un lien était possible, mais aucune API était disponible, et il a donc fallu changer le MCD en conséquence.

Néanmoins, ayant effectué des recherches en amont, j'ai pu anticiper ce risque et prévoir au cas-où sur les maquettes des pages de création d'article, ce qui fait qu'au final j'ai perdu peu de temps sur ce problème.

→ MCD - Annexes

Intégration

Mais avant d'implémenter la base de données, il m'a fallu choisir les technologies pour réaliser le système de commandes : au départ je voulais rester sur du Wordpress en pensant que ce n'était pas réalisable autrement.

Wordpress, avec des extensions comme WooCommerce, qui permettent facilement la création d'un site marchand, reste extrêmement rigide, et tous ces systèmes comporte obligatoire le paiement, ce qui était contre le cahier des charges du client.

Finalement, mon choix s'est porté sur Symfony, un **framework*** qui nous a été présenté lors du Semestre 4, avec qui j'étais bien plus familier, et qui avec l'ensemble de ses composants, permet une gestion de base de données et d'utilisateurs.

Intégration

Après avoir initié le projet Symfony et mis en place la base de données, j'ai commencé par intégrer les pages du site à partir des maquettes.



Aperçu de la page d'accueil

Pour créer les pages, je me suis servi du moteur de templates **Twig***, qui associé à Symfony permet de mettre en place des pages en HTML associé à de la logique en PHP bien plus facilement. J'ai aussi eu recours au framework TailwindCSS qui accélère la création des pages sans avoir à toucher à un seul fichier CSS. Il permet aussi une approche **mobile-first***, qui était une demande du client .

Comme ce n'était pas la première fois que je me servais de ces outils, et étant déjà très familier avec le développement **frontend***, cette partie du projet s'est déroulé sans problèmes et très rapidement. Néanmoins, la méthode à suivre en temps normal est de commencer par le **backend*** car c'est bien plus pratique, heureusement ça n'a pas posé plus de problèmes sur ce projet.

Gestion des données

Pour le backend, j'ai décidé de commencer par la gestion des utilisateurs, précisément faire un CRUD.

Symfony est très puissant et efficace pour ça, il y a des commandes qui le font automatiquement, avec un système d'inscription, de connexion et de sécurité efficace. J'ai perdu du temps au départ car je ne m'étais pas assez renseigné sur le sujet, et j'ai voulu le faire de zéro.

Ensuite il faut garantir à l'utilisateur la pleine gestion et maîtrise de ses propres données, Symfony est aussi efficace pour créer des formulaires et des routes qui gèrent ces cas.

Le plus long était de suivre la documentation officielle et de tester petit à petit. Le plus difficile de trouver une solution pour des cas plus spécifiques, comme la gestion des rôles qui utilisent des enums, ou des clés étrangères dans la suppression d'un utilisateur.

```
#[Route("/profile", name: "profile")]
public function profile (OrderRepository $orderRepository, OrderInfosService $orderInfosService): Response
{
    $user = $this->getUser();

    $orders = $orderRepository->findBy([
        'user' => $user,
    ]);

    foreach ($orders as $order) {
        $order->total = $orderInfosService->getTotalPrice($order->getId()); // Calcule le total de la commande
        $orderInfosService->testState($order); // Vérifie si l'état de la commande correspond toujours à la commande
        $order->isModifiable = $orderInfosService->isModifiable($order); // Vérifie si la commande est toujours modifiable
    }

    return $this->render('client/profile.html.twig', [
        'user' => $user,
        'orders' => $orders,
    ]);
}
```

Exemple de code pour l'affichage de l'utilisateur

Compte de test
test.test@test.test
0641976084
Endroit: Lorem ipsum dolor sit amet consectetur.

Modifier les informations
Se déconnecter
Supprimer mon compte

Historique de commandes

Pour le 27 mai 2025
Commande le 27 mai 2025
28,00€ Voir la commande Terminé

Pour le 12 juin 2025
Commande le 12 juin 2025
61,60€ Voir la commande Terminé

Page du profil

Gestion des données

Puis j'ai enchaîné sur la gestion des articles, qu'il fallait pouvoir créer, afficher, modifier et supprimer aussi.

La plus grande difficulté était comment gérer les images des articles. J'ai cherché plusieurs solutions sur des tutos ou des forums mais peu correspondaient à l'architecture du projet.

À plusieurs reprises dans le code, je suis tombé sur des erreurs que je ne savais pas résoudre, et ne trouvant aucune réponse sur des forums comme StackOverflow, j'ai fini par avoir eu recours à l'IA. J'ai surtout utilisé l'IA Claude, qui se voulait être sécurisée et éthique, et s'est avéré être un outil intéressant, surtout dans la résolution de problèmes précis et complexes, mais n'étais pas toujours très précis.

```
TP <!--Summary Section-->
<section class="w-full flex flex-col items-center gap-6 px-8 py-5 text-left lg:col-span-4">
  {% if order %}
    <h3 class="w-full">Récapitulatif</h3>
    <h5 class="w-full">Pour le {{ order.date|format_datetime('medium', 'none', locale: 'fr') }}</h5>
    <input type="date" min="{{ 'now'|date_modify('+7 days')|date('Y-m-d') }}">
    <div class="w-full flex flex-col gap-5 py-5 border-t-2 border-gray">
      {% set total = 0 %}
      {% for article in articles %}
        <!--Price Div-->
        <div class="w-full flex justify-between">
          <p>{{ article.articleid.name }}</p>
          {% set price = article.articleid.price|default(0) %}
          {% set quantity = article.quantity|default(0) %}
          {% set total = total + price * quantity %}
          <p>{{ quantity }}</p>
          <p>{{ (price * quantity)|number_format(2, ',') }}€</p>
        </div>
      {% endfor %}
    <div class="w-full flex justify-between py-4 border-t-2 border-dgray">
      <p>Total</p>
      <p>{{ total|number_format(2, ',') }}€</p>
    </div>
    <div>
      <a href="{{ url('basket_validate', {'id': order.id}) }}" class="w-fit h-fit px-6 py-3 rounded-sm bg-red text-white shadow-box">Valider la commande</a>
      <a href="{{ url('search', {'id': 1}) }}" class="w-auto h-auto px-6 py-3 rounded-sm bg-white border-2 border-red text-red shadow-box">Continuer à commander</a>
    </div>
  {% endif %}

  J'ai 2 input dans le lot, en gros je voudrais savoir quelle méthode il faudrait utiliser pour pouvoir mettre à jour la date de l'Order ainsi que la quantité de l'Orderitem si y a eu une modification
```

Exemple de **prompt*** envoyé à l'IA

Gestion des données

La partie la plus longue du backend était la gestion des réservations des utilisateurs, car c'est la fonction principale du système.

Les commandes se découpent en plusieurs petites fonctionnalités toutes différentes et qui demandent toutes des vérifications et de la sécurité : créer un panier, le modifier, le valider, le supprimer, changer une commande validée, chercher et valider ou non les commandes côté admin.

Je n'ai pas rencontré de problèmes, le plus difficile était de tester chaque fonctionnalité et essayer de les "casser" pour prévoir chaque problème. Je n'avais pas clairement inclus cette tâche dans le planning, donc j'ai perdu du temps dessus qui aurait dû servir à d'autres tâches.

Et pendant la programmation, je me suis rendu compte qu'il manquait des détails ou des fonctionnalités, comme prévoir une date pour la commande, qu'il fallait ajouter sur le tas.

Gestion des données

Enfin, le dernier grand bloc de backend à produire était l'interface administrateur. Le système doit permettre de commander, mais aussi à l'entreprise d'accepter leurs futurs utilisateurs ou valider des commandes.

Symfony permet aussi facilement de bloquer les routes sensibles aux utilisateurs normaux, il n'y a donc pas eu de problème de sécurité. Et vu que j'étais bien plus à l'aise avec le framework et que les fonctionnalités principales étaient déjà implémentées, la partie admin s'est déroulée plus rapidement que prévu.

```
#[Route("/admin/accounts/validate/{id}", name: "admin_accounts_validate", requirements: ["id" => "[0-9-]+"])]
public function adminAccountsValidate (Request $request, UserRepository $userRepository, EntityManagerInterface $entityManager)
{
    $id = $request->attributes->get('id');
    $user = $userRepository->findOneBy([
        'id' => $id,
    ]);
    if (!$user) {
        throw $this->createNotFoundException('l\'utilisateur n\'existe pas');
    }

    $user->validateUser();
    $entityManager->flush();

    try {
        $email = (new Email())
            ->from('contact@lacvi.fr')
            ->to($user->getEmail())
            ->subject('Compte validé')
            ->text('Votre compte sur le site Internet de LACVI a été validé.');
```

Code pour valider un compte

```
#[Route("/admin/article/add", name: "admin_article_add")]
public function addArticle (Request $request, EntityManagerInterface $entityManager, FileUploadService $fileUploadService): Response
{
    $article = new Article();
    $form = $this->createForm(ArticleForm::class, $article);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        $article->setStock(1);

        $file = $form->get('image')->getData();

        if ($file) {
            try {
                $fileName = $fileUploadService->upload($file);
                $article->setImage($fileName);
            } catch (FileException $e) {
                $this->addFlash('error', 'Erreur lors de l\'upload de l\'image');
            }

            return $this->render('admin/addarticle.article.html.twig', [
                'articleForm' => $form,
            ]);
        }

        $entityManager->persist($article);
        $entityManager->flush();
        $this->addFlash('success', 'l\'article a été ajouté avec succès');

        return $this->redirectToRoute('admin_index');
    }

    return $this->render('admin/addarticle.html.twig', [
        'articleForm' => $form,
    ]);
}
```

Code pour créer un article

Mails

Un autre des points du cahier des charges du projet était qu'il fallait que l'entreprise reçoive un mail pour prévenir de la création d'un compte, validation d'une commande ou de l'envoi d'un mail de contact.

La création d'un mail dans le code dans Symfony est relativement simple, le plus difficile est de mettre en place le système qui envoie les mails. Cette partie est très différente du reste du code et très complexe, car il y a beaucoup de technologies différentes et beaucoup ne sont pas évidentes à mettre en place.

Après beaucoup d'essais, je suis parvenu à trouver un tuto en français efficace, qui utilise Mailtrap comme boîte-mail de test. Il a fallu un temps d'adaptation toutes les autres éventuelles erreurs et les problèmes techniques, puis lier chaque mail automatique à chaque action.

```
#[Route("/contact", name: "contact")]
public function contact(Request $request, MailerInterface $mailer): Response
{
    $form = $this->createForm(ContactForm::class);
    $form->handleRequest($request);

    if ($form->isSubmitted() && $form->isValid()) {
        try {
            $user = $this->getUser();
            $email = (new Email())
                ->from('contact@lacvi.fr')
                ->to('contact@lacvi.fr')
                ->subject($form->get('subject')->getData())
                ->text('Message envoyé de '. $user->getName() . ' : '. $form->get('message')->getData());

            $mailer->send($email);
            $this->addFlash('success', 'Mail envoyé');
        } catch (\Exception $e) {
            $this->addFlash('error', 'Erreur lors de l\'envoi : ' . $e->getMessage());
        }

        return $this->redirectToRoute('home');
    }

    return $this->render('client/contact.html.twig', [
        'contactForm' => $form,
    ]);
}
```

Code pour envoyer un mail de contact

Création de PDF

Enfin, la dernière fonctionnalité à implémenter dans le site était de pouvoir créer un devis en .pdf de la commande.

J'ai vite trouvé un tuto vidéo qui expliquait facilement comment le faire, le plus difficile était de faire la mise en page du devis, car il n'y a pas toutes les lignes de CSS qui fonctionne dessus.

Au final, l'essentiel fonctionne sans problème, et son apparence ne gêne pas le client car c'était au départ pensé comme une facture temporaire.

→ Projet Symfony - Annexes

Commande N°

- Date :
- Réf de Cde :
- Téléphone :

- 16
- 12 juin 2025
- Monsieur Timéo
- 0641976584

Compte de test

Endroit - Lorem ipsum dolor sit amet consequr

BON de COMMANDE

Ce bon de commande est temporaire et à titre indicatif. Un futur devis sera fourni par la suite.

Désignation	Prix unitaire TTC	Quantité	Total TTC
Tarani Rosé	5,60€	3	16,80€
Tarani Blanc	5,60€	8	44,80€

Total TTC : 61,60€

Lieu de vente : LACVI - Beaumont

CONDITIONS GENERALES DE VENTE

(1) Sauf convention particulière aucun escompte n'est consenti pour paiement comptant. (2) Tout retard dépassant 30 jours entrainera des pénalités au taux du découvert bancaire. (3) Aucun échange ni retour ne sera accepté sauf accord préalable. (4) Les produits objets de votre commande restent notre entière propriété jusqu'à paiement intégral de la facture. (5) Pour tout litige concernant les présentes conditions générales de vente,

Exemple de devis PDF

Conclusion

04

À la date de fin du stage, le 20 Juin 2025, je n'ai pas hébergé et rendu le site comme convenu dans le cahier des charges. J'ai perdu beaucoup de temps sur la partie backend du projet à cause d'une mauvaise planification des tâches et un manque d'efficacité, peut-être que la mission au global était aussi difficile à réaliser seul dans le temps imparti.

Heureusement, les clients sont compréhensifs, et le rendu tardif du site ne met pas en péril leur entreprise, cependant je suis conscient que dans un cadre normal, ce scénario est à éviter à tout prix.

Néanmoins, je tire à l'issue de ce stage une expérience enrichissante, où j'ai pu tester seul mes compétences en développement Web avec un cadre et un enjeu réel.

J'ai été très bien accueilli chez LACVI, qui ont été des clients très agréables et précis sur leur demande.

Ce projet a permis de consolider mes compétences en développement, design, mon travail en autonomie ainsi que ma démarche de travail. Ce stage m'a rassuré sur le fait que je suis bien capable de réaliser un projet de cette envergure seul, bien qu'il me reste enfin du chemin à faire.

Et en tant qu'expérience professionnelle de taille, ce stage est une ligne utile sur mon CV, dans le cadre de la recherche de ma future alternance.

Glossaire

05

API (Interface de Programmation d'Application) : C'est une interface qui permet à deux programmes de communiquer entre eux

Arborescence : Organisation hiérarchique des pages d'un site web

Backend : Partie cachée d'un site ou d'une appli qui gère les données, les utilisateurs, les règles métiers, souvent reliée à une base de données

CRUD : Opérations de base pour les utilisateurs : Créer (Create), Lire (Read), Mettre à jour (Update) et Supprimer (Delete) leurs données

Framework : Ensemble de composants logiciels réutilisables qui permettent de développer de nouvelles applications plus efficacement

Frontend : Partie visible par l'utilisateur, affichée dans le navigateur, c'est l'interface graphique d'un site (pages, boutons, menus, animations...)

Maquette : Image ou document qui montre à quoi ressemblera une page web finie, avec les couleurs, textes, images

Glossaire

MCD (Modèle Conceptuel de Données) : Schéma pour représenter les données et leurs relations sans se soucier de la technique

MPD (Modèle Physique de Données) : Version technique du MCD, prête à être transformée en base de données

Méthode agile : Façon de gérer un projet en le découpant en petites étapes courtes (sprints), avec des retours fréquents; qui favorise l'adaptation

Mobile-first : Approche de design qui consiste à créer une interface optimisée d'abord pour smartphone, puis à l'adapter aux écrans plus grands

Prompt : Instruction ou question donnée à une IA pour qu'elle réponde ou génère un contenu

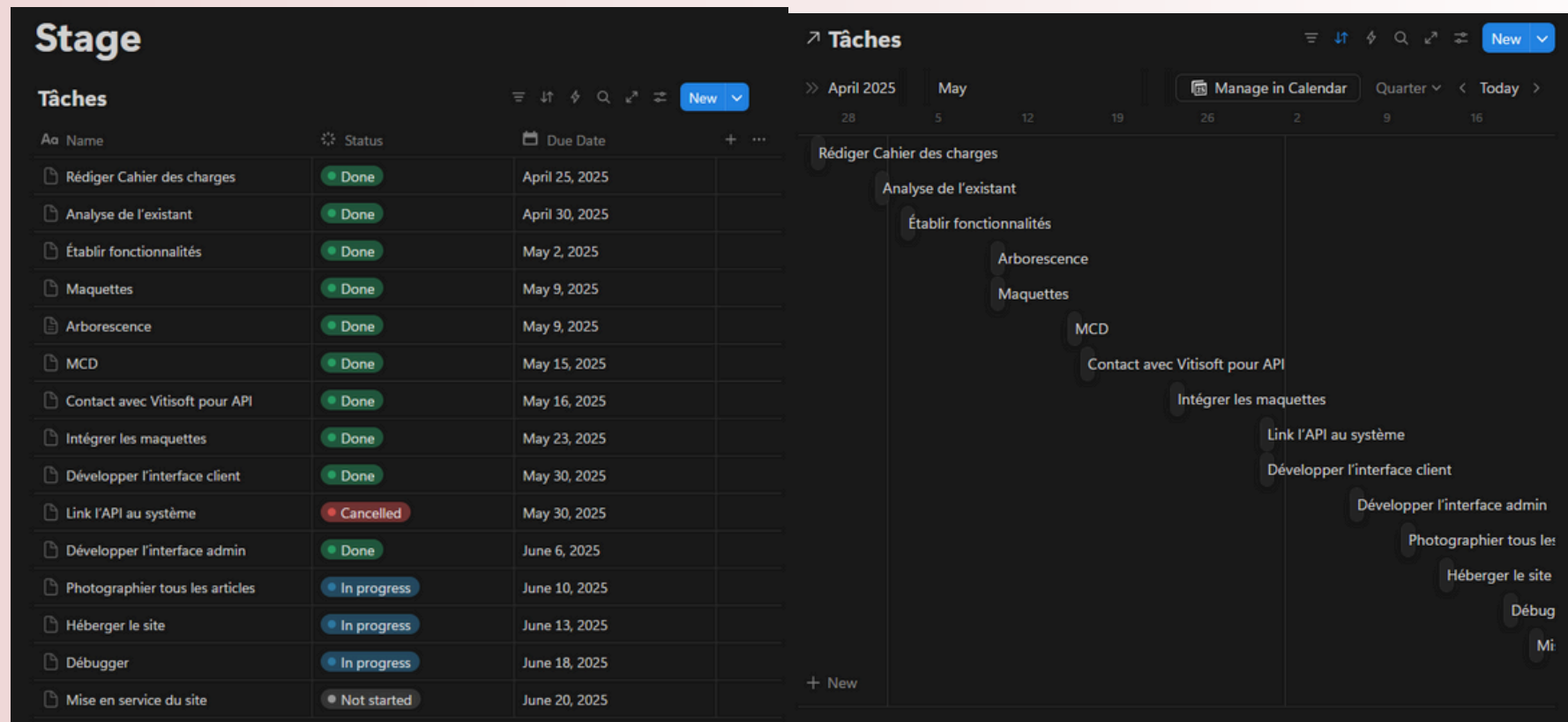
Twig : Langage de template utilisé pour générer des pages HTML dynamiques, souvent avec Symfony, qui permet d'insérer facilement des données dans des pages web

Wireframe : Schéma simple et sans style d'une page web. Il montre les zones principales (menu, texte, images) pour planifier l'interface avant le design graphique

Annexes

06

N°1 - Planning des tâches



N°2 - Cahier des charges

Cahier des charges

Objectif : Création d'un système de commande en ligne pour les associations et les clients de LACVI

Spécifications techniques :

- Le site doit être accessible sur smartphone et intuitif pour les clients.
- Le système complet doit respecter les normes de disponibilité, de sécurité et d'accessibilité.

Fonctionnalités demandées :

- Interface administrateur
 - L'interface doit permettre de réaliser les opérations suivantes :
 1. Valider ou non les commandes des clients
 2. Accéder aux commandes en cours
 3. Modifier les informations des produits
 4. Modifier le contenu de la page de renseignements
 - Modifier la base de données associée au site.
 - Pour directement accéder aux stocks, le système doit être lié à l'API de Vitisoft si possible.

- Interface client

- Système d'identification des clients ainsi que de création de compte, directement par l'utilisateur ou par l'administrateur.
- Affichage des produits disponibles, leur quantité et leurs images.
- Envoi d'un email de confirmation après la commande d'un client.
- Page « À propos ».
- Affichage de l'historique de commandes.
- Système de mise à jour des informations de l'utilisateur (CRUD).
- Création d'une facture PDF lors de la validation de la commande.

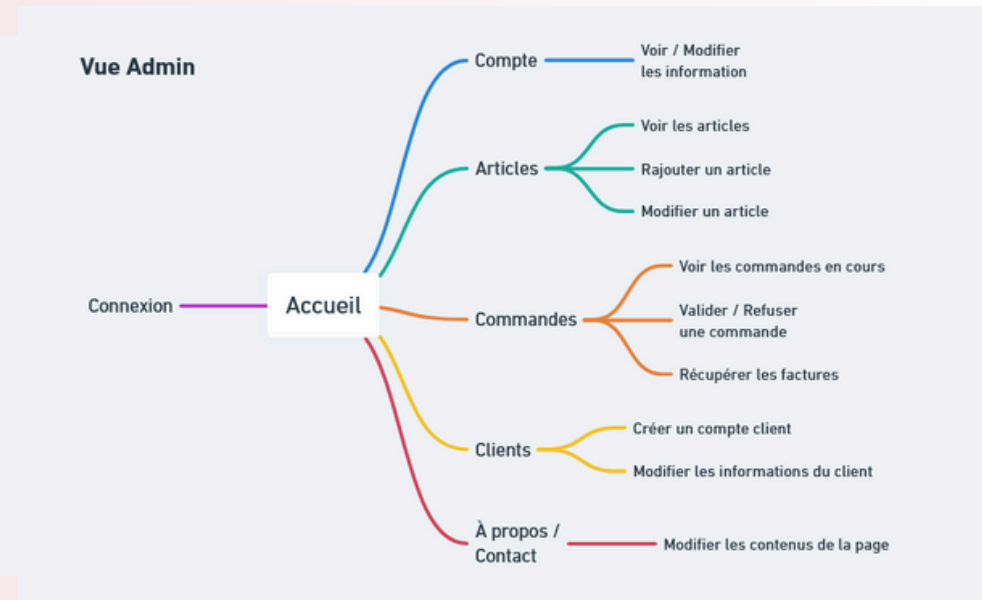
Navigation :

- Le design du site web doit être clair, respecter la charte graphique de LACVI et les normes d'accessibilité.
- Le système doit être performant et rapide pour garantir une navigation fluide.
- Le site web doit fournir une expérience utilisateur adaptée à la cible.

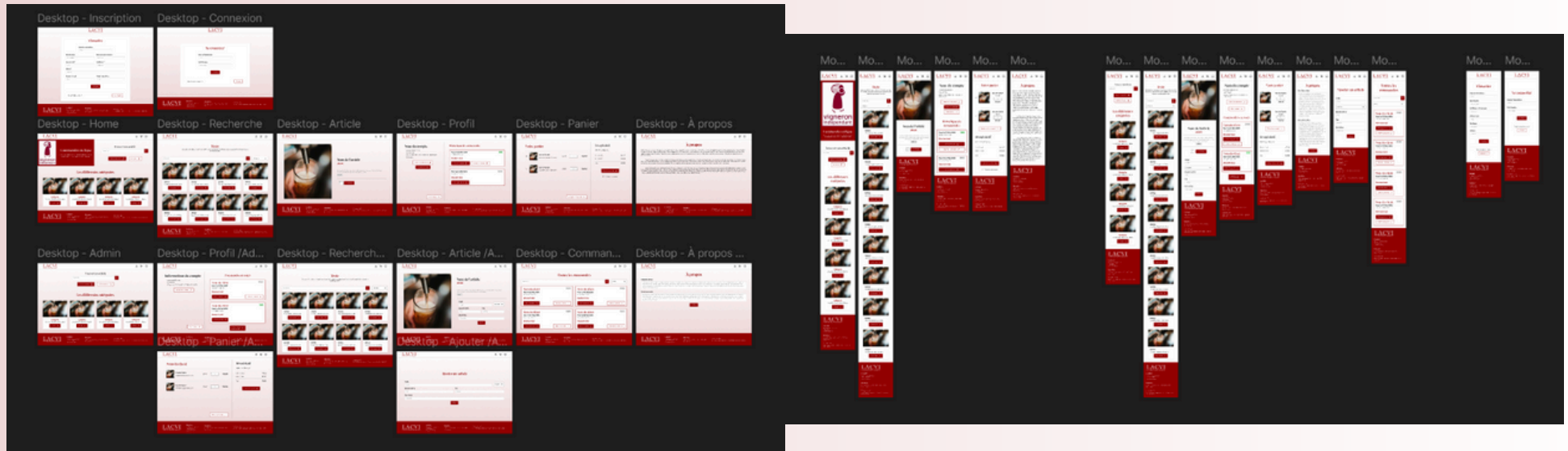
Livraison de la demande :

- Le site doit être terminé, testé, débogué et hébergé au plus tard pour la fin du stage : **20 Juin 2024**.

N°3 - Arborescence

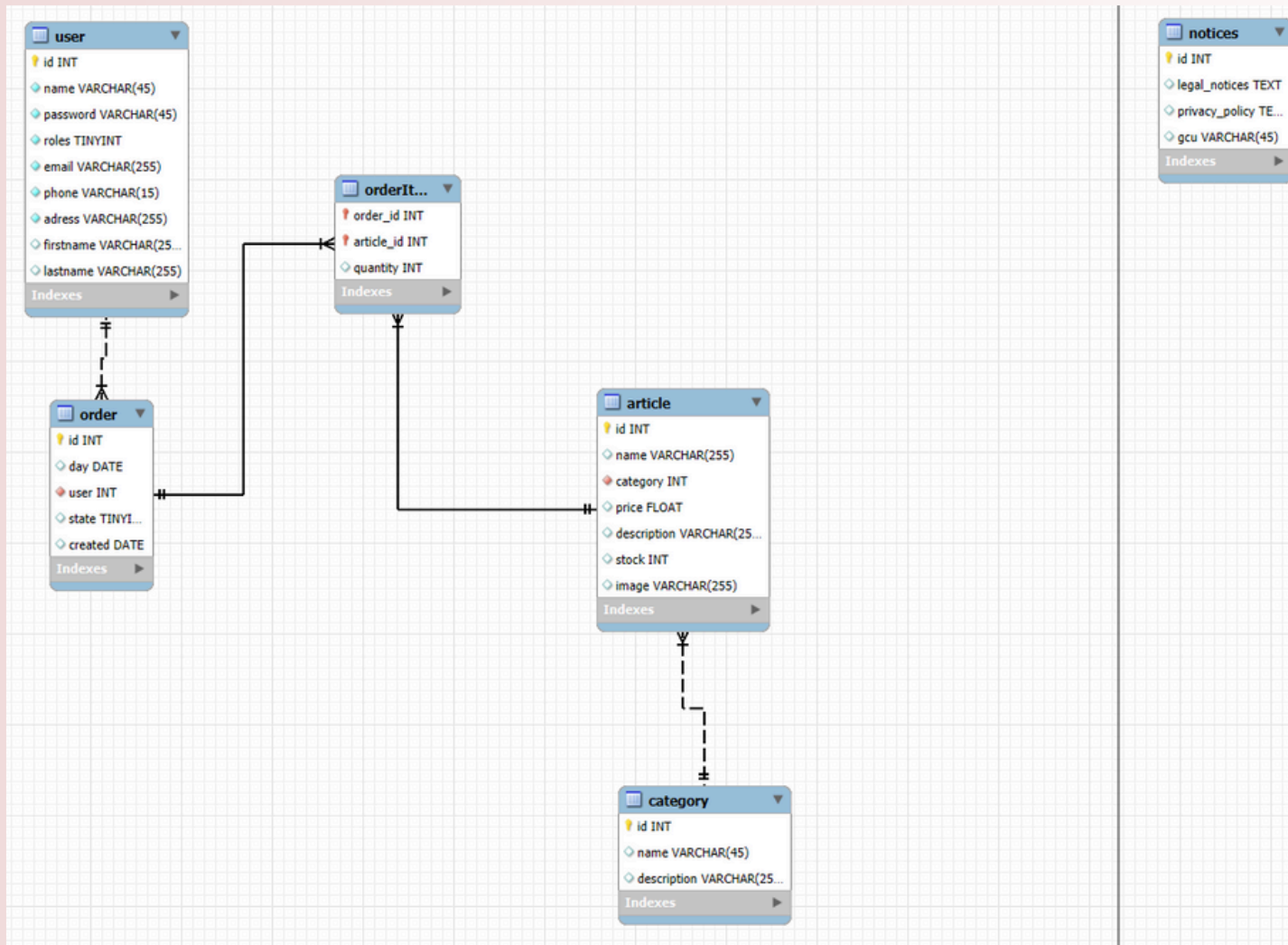


N°4 - Maquettes



[Lien vers la maquette Figma](#)

N°5 - MCD



N°6 - Projet Symfony

The screenshot shows a GitHub repository named 'stage_lacvi' (Private). The repository is at the 'main' branch, has 1 branch, and 0 tags. The file structure is as follows:

File/Folder	Commit	Time
assets	first commit	19 hours ago
bin	first commit	19 hours ago
config	first commit	19 hours ago
migrations	first commit	19 hours ago
node_modules	first commit	19 hours ago
public	first commit	19 hours ago
src	first commit	19 hours ago
templates	first commit	19 hours ago
tests	first commit	19 hours ago
.gitignore	first commit	19 hours ago
importmap.php	first commit	19 hours ago
package-lock.json	first commit	19 hours ago
package.json	first commit	19 hours ago
phpunit.xml.dist	first commit	19 hours ago
symfony.lock	first commit	19 hours ago

The repository has 1 commit (6132d1a) made 19 hours ago. The 'About' section indicates no description, website, or topics are provided. The 'Releases' section shows no releases published. The 'Packages' section shows no packages published. The 'Languages' section shows a bar chart with the following data:

Language	Percentage
PHP	53.6%
Twig	32.7%
CSS	11.4%
JavaScript	2.3%

The 'Suggested workflows' section shows a workflow for Node.js, with a 'Configure' button.

[Lien vers le repository GitHub](#)

